

Examak: An AI-Enhanced Web-Based Examination System

Mahmoud Khairalla Amin Morsi

Department of Informatics Engineering, Faculty of Engineering

Universitas Nusa Putra

Jl. Raya Cibolang No.21, Cisaat, Sukabumi, West Java 43152, Indonesia

mahmoud.khairalla_ti23@nusaputra.ac.id

Abstract—Examak is an AI-enhanced web-based examination system designed to address the growing need for automation, reliability, and scalability in modern educational assessments. The primary objective of this work is to deliver a secure, efficient, and intelligent platform that facilitates the creation, management, and grading of exams, focusing on objective question types such as multiple-choice and true/false items. Traditional examination workflows demand substantial manual effort from instructors for paper preparation, distribution, and evaluation, and are prone to human error and bias. In contrast, Examak minimizes manual intervention by leveraging rule-based intelligent algorithms for automatic grading, dynamic question randomization, and a structured workflow for exam scheduling, student management, and result reporting. The platform is built on the MERN stack—MongoDB, Express.js, React.js, and Node.js—and adopts a three-tier architecture comprising presentation, application, and data layers. It incorporates JWT-based role-based access control (administrator, teacher, student), a dynamic question bank, and real-time performance analytics. Experimental evaluation demonstrates 100% grading accuracy for objective items, average login and dashboard response times near 500 ms, exam submission latencies below 1.5 s, and stable performance with up to 300 concurrent users. The architecture is explicitly designed to be AI-ready, allowing future extensions such as NLP-based question generation, automated essay grading, and AI-assisted proctoring through facial recognition and behavior analysis.

Index Terms—AI-enhanced examination, MERN stack, automated grading, e-learning, role-based access control, online assessment, natural language processing, web application, educational technology.

I. INTRODUCTION

A. Background and Motivation

The transformation of education from traditional, paper-based methods to digital, technology-driven platforms has accelerated dramatically over the past decade. The COVID-19 pandemic, in particular, exposed the fragility of traditional assessment workflows and forced institutions worldwide to adopt remote examination solutions almost overnight [1], [2]. Despite this widespread digital shift, a significant portion of educational assessment is still managed through manual or semi-digital processes that are error-prone, slow, and difficult to scale.

Traditional examination workflows require instructors to invest considerable effort in preparing question papers, distributing them, supervising sessions, and grading answer sheets manually. These activities are not only time-consuming but also introduce variability, fatigue-driven errors, and unconscious bias into the evaluation process. With increasing class sizes and the global shift toward blended and remote learning models, the limitations of these traditional approaches have become a serious bottleneck for educational institutions.

Beyond the operational burden, traditional paper assessment also offers little to no analytical insight into student learning patterns. Instructors must invest additional manual effort to compile statistics, identify weak topics, or detect anomalies in performance. As institutions move toward evidence-based and data-driven instructional design, this lack of native analytical capability becomes a serious limitation. Furthermore, the asynchronous and geographically distributed nature of modern higher education—particularly in developing regions of Southeast Asia and the Middle East—makes any system that depends on physical co-location intrinsically fragile. A purely web-based, automated, and AI-ready platform is therefore not merely a convenience, but a structural requirement for sustainable educational delivery.

B. Problem Statement

Existing digital examination tools and learning management systems (LMSs) such as Moodle, Blackboard, ExamSoft, and Google Forms each suffer from specific limitations. Moodle, while powerful, demands substantial technical expertise for deployment and ongoing administration. Google Forms is easy to use but lacks robust authentication mechanisms, secure session handling, and any meaningful protection against cheating. Commercial platforms such as Blackboard and ExamSoft offer comprehensive features but come with

high licensing costs that are prohibitive for small and medium institutions, especially in developing regions [3].

A second class of problems concerns architectural rigidity. Most legacy examination platforms were not designed with modern AI components in mind. Retrofitting NLP-based question generation, computer-vision proctoring, or behavioral analytics into a monolithic legacy stack is expensive and risky. A new generation of platforms must therefore be designed from the start with explicit, well-typed extension points for AI services, while still preserving the simplicity and robustness expected of a production examination tool.

There is therefore a clear need for a *lightweight, secure, affordable, and intelligent* examination platform that combines focused exam-lifecycle features with modern web technology and a forward-looking architecture capable of incorporating artificial intelligence (AI) modules over time. The work presented in this paper directly addresses this gap.

C. Objectives

The principal objectives of this work are:

(1) to design and implement a secure web-based examination platform that automates the creation, scheduling, delivery, and grading of objective examinations; (2) to enforce role-based access control (RBAC) for administrators, teachers, and students; (3) to provide a responsive, intuitive interface that minimizes cognitive load and supports a wide range of devices; (4) to deliver real-time performance analytics for both instructors and learners; and (5) to architect the system in a modular fashion such that future AI components—NLP-driven question generation, automated essay evaluation, and intelligent proctoring—can be plugged in without architectural rework.

D. Contributions

The contributions of this paper are summarized as follows:

(1) A complete full-stack implementation of Examak using the MERN stack, integrating React.js, Node.js, Express.js, and MongoDB with JWT and Bcrypt-based security primitives. (2) A deterministic rule-based auto-grading engine that achieves 100% grading accuracy for objective question types under controlled testing. (3) A modular three-tier architecture that cleanly separates presentation, business logic, and data layers and explicitly accommodates an AI extension layer interfaced via a Python Flask micro-service. (4) Empirical performance evaluation demonstrating sub-second core operations and stable behavior up to 300 concurrent users. (5) A comparative analysis with widely-used LMSs that positions Examak as a lightweight, secure, and AI-ready alternative.

E. Paper Organization

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the proposed system, including the architecture and UML/DFD models. Section IV details the implementation. Section V describes the testing methodology and reports performance results. Section VI discusses the findings and limitations, and Section VII concludes the paper and outlines future work.

II. RELATED WORK

A. Existing E-Examination Platforms

Several digital assessment platforms have been widely adopted in academia and industry. *Moodle* is a well-known open-source LMS that provides extensive features for course management, quizzes, and grading [7]. However, its setup, configuration, and maintenance require significant technical expertise, and its user interface can be overwhelming for non-technical instructors and learners.

Google Forms [8] offers a lightweight approach to quiz creation but lacks robust authentication, secure session management, role-based access control, and dedicated mechanisms to prevent cheating. *Blackboard* and *ExamSoft* are powerful proprietary platforms used by large universities and certification bodies, yet they incur substantial licensing costs and offer limited customizability for smaller institutions or research-oriented deployments.

B. Academic Research

Several studies underline the need for modern, automated assessment tools. Ali *et al.* [4] argue that faster, automated grading combined with immediate feedback materially improves student learning. Kumar and Singh [5] show that real-time progress analytics support better concept retention. Chen *et al.* [6] demonstrate that interface simplicity strongly influences online learning performance, especially for students with limited access to high-end hardware. Zhang *et al.* [3] discuss the use of browser/server (B/S) architectures for online examinations and highlight scalability and maintainability benefits. Al-Azawei *et al.* [1] and Eke [2] examine barriers and opportunities for e-learning adoption in developing regions and identify cost, complexity, and security as recurring concerns.

C. Identified Gaps

From the literature and existing-systems analysis, four recurring gaps emerge: (i) high deployment and maintenance complexity (*Moodle*, *Blackboard*); (ii) high licensing cost (*Blackboard*, *ExamSoft*); (iii) weak security and limited cheating mitigation (*Google Forms*); and (iv) shallow performance analytics that fall short of supporting data-driven instructional decisions.

D. Positioning of Examak

Examak is positioned as a lightweight, focused, and AI-ready alternative. Rather than competing with full-featured LMSs, it concentrates on the examination lifecycle: secure authentication, dynamic exam authoring, randomized

delivery, automated grading, and analytics. The system is designed around the MERN stack to maximize developer accessibility, deployment simplicity, and forward compatibility with modern web technologies. A side-by-side comparison with major existing platforms is given in Table I.

TABLE I. COMPARISON OF EXAMAK WITH EXISTING SYSTEMS

Feature	Moodle	Google Forms	Blackboard	Examak
Open Source	Yes	No	No	Yes
Licensing Cost	Free	Free	High	Free
Auto-Grading (Objective)	Yes	Partial	Yes	Yes (100%)
Role-Based Dashboards	Partial	No	Yes	Yes
Question Randomization	Yes	Limited	Yes	Yes
AI-Ready Architecture	No	No	Partial	Yes
Lightweight Setup	No	Yes	No	Yes
NLP Question Generation	No	No	No	Planned
Real-Time Analytics	Partial	Limited	Yes	Yes
Anti-Cheating Controls	Yes	No	Yes	Yes

III. PROPOSED SYSTEM

A. System Overview

Examak follows a classical three-tier architecture that separates concerns across the presentation, application, and data layers. The presentation layer is a React.js single-page application that renders responsive role-specific dashboards. The application layer is a Node.js/Express.js back-end exposing RESTful endpoints for authentication, exam management, grading, and analytics. The data layer is implemented in MongoDB and accessed via the Mongoose ODM, providing schema validation and indexed lookups.

This separation enables independent scaling of front-end and back-end components, simplifies maintenance, and provides a natural boundary at which the Python-based AI services (Section III.D) can be attached via Flask micro-services. Each tier communicates with the next through narrow, well-typed contracts: the front-end interacts with the back-end exclusively through versioned RESTful HTTPS endpoints, and the back-end interacts with the database exclusively through Mongoose schemas. This explicit contract discipline simplifies debugging, version migration, and security audits.

The three-tier model was preferred over a monolithic design or a micro-services explosion for three reasons. First, it preserves the simplicity expected of a focused examination platform. Second, it keeps the operational footprint small enough for deployment on modest cloud infrastructure, which

matters for institutions with limited budgets. Third, it leaves a clean seam between business logic and AI: the AI module can evolve from a Python prototype to a containerized service or even a managed cloud endpoint without changing any other layer of the system.

B. Architecture Layers

Presentation Layer. The front-end is implemented in React.js using functional components and hooks. Styling is performed with Tailwind CSS for utility-first design and consistency across devices. HTTP communication with the back-end is handled via Axios with interceptors that automatically attach JWT tokens and refresh sessions on expiry.

Application Layer. The back-end uses Node.js with Express.js. It implements JWT-based authentication, Bcrypt password hashing, RBAC middleware for protecting routes, dynamic exam scheduling, the auto-grading engine, and structured logging. RESTful endpoints follow resource-oriented conventions (e.g., `/api/exams`, `/api/users`, `/api/results`).

Data Layer. MongoDB is used to persist users, subjects, questions, exams, attempts, answers, and reports. Mongoose schemas enforce structure and validation; indices on user identifiers, exam identifiers, and timestamps optimize the most common query paths.

C. System Modeling

A complete UML and DFD model was produced before implementation. Fig. 1 shows the Entity-Relationship Diagram (ERD), capturing the core entities and their relationships: students take exams, teachers (referred to as "doctors" in some academic contexts) create exams and questions, exams cover subjects, questions belong to exams, students provide answers, and reports aggregate student grades. The ERD also reserves an entity for the AI-generated report module, anticipating future intelligent analytics.

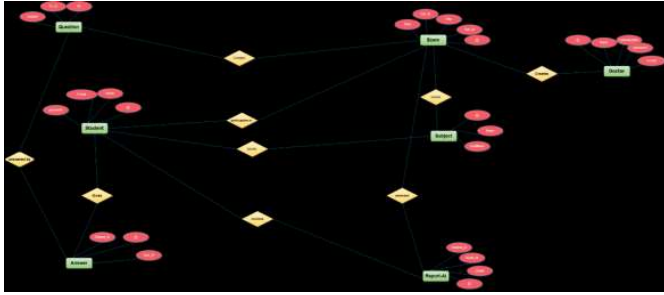


Fig. 1. Entity-Relationship Diagram (ERD) of the Examak database, showing core entities (Student, Teacher, Exam, Subject, Question, Answer, Report-AI) and their relationships.

Fig. 2 presents the Data Flow Diagram at Level 0 (context diagram). It treats the Examak system as a single process and shows the three principal external entities—Students, Teachers, and the AI-driven Report module—together with the data flows that cross the system boundary.



Fig. 2. Data Flow Diagram Level 0 (context diagram) of the Examak system, illustrating its high-level boundary with external actors.

Fig. 3 decomposes the system into its principal sub-processes (Level 1). It shows how questions are submitted, stored, assigned to instructors, answered, evaluated, and how the corresponding reports are generated.

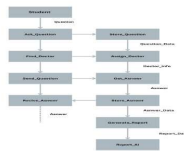


Fig. 3. Data Flow Diagram Level 1, decomposing Examak into the question-management, assignment, answering, and report-generation sub-processes.

Fig. 4 shows the UML Use Case Diagram. The principal actors are the Student and the Teacher, with an implicit Administrator role for system management. Use cases include Login (with credential verification), View Exam Schedule,

Take Exam, View Results, Manage Subjects, Create Exam, Manage Users, and View Student Performance.

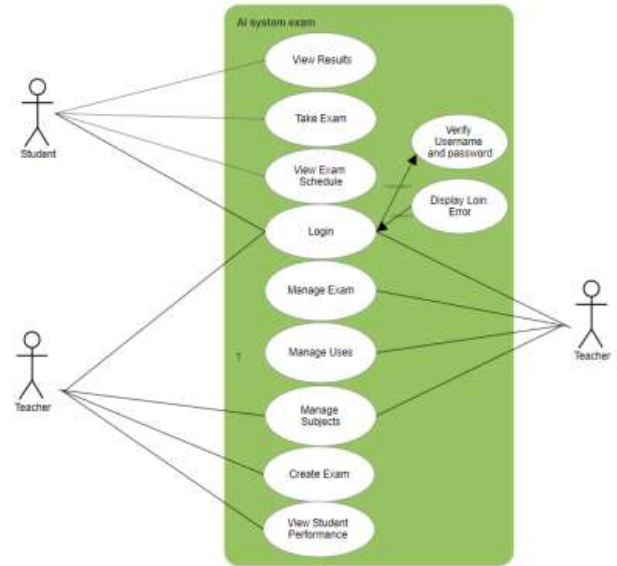


Fig. 4. UML Use Case Diagram showing the principal actors (Student, Teacher) and the use cases they can invoke.

Fig. 5 presents the UML Sequence Diagram for the most important interaction flows—login, exam creation by an instructor, exam attempt by a student, automated grading, and result publication. The diagram explicitly models the database and grading subsystems as separate lifelines, which clarifies the boundary between business logic and data persistence.

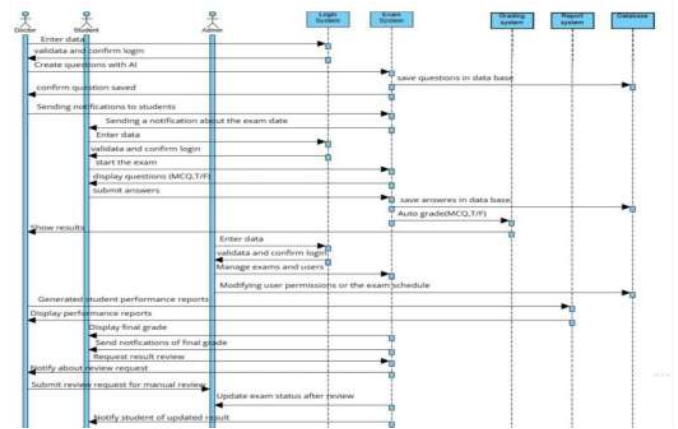


Fig. 5. UML Sequence Diagram showing the chronological interaction between users and the login, exam, grading, reporting, and database subsystems.

Finally, Fig. 6 shows the Activity Diagram for the login and role-based navigation flow. Following a successful login, the user is dispatched to a role-specific home page (Administrator, Teacher, or Student) and from there to the appropriate workflows: managing the system, creating and submitting exams, or taking exams and viewing results.

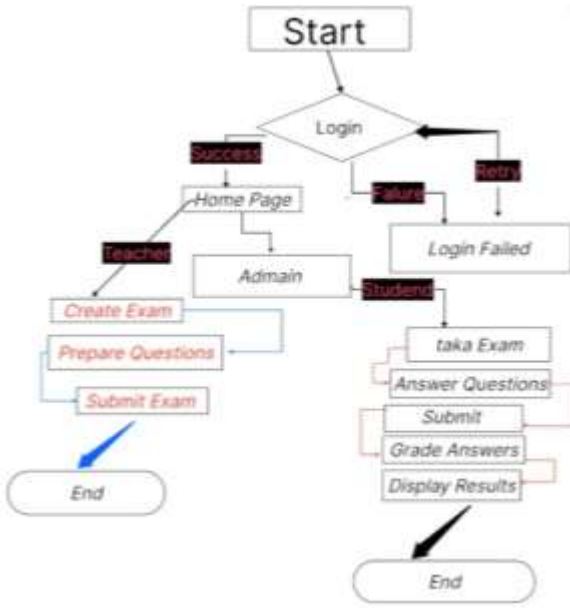


Fig. 6. Activity Diagram for the login process and role-based navigation across Administrator, Teacher, and Student paths.

D. Algorithms and Frameworks

Examak relies on four core algorithmic components. The first is a deterministic *rule-based auto-grading engine* (Algorithm 1) that compares student responses to the canonical correct answers stored in the database and aggregates a weighted score. The second is *Fisher-Yates shuffling* (Algorithm 2) for question randomization, ensuring that each student receives a unique question order without statistical bias [13]. The third is *JWT-based RBAC middleware* [14], which authenticates each request and enforces role-specific authorization before dispatching to controllers. The fourth is the *AI extension interface*, a Flask-based Python micro-service that exposes endpoints for NLP-based question generation and future essay grading.

Algorithm 1. Auto-Grading Engine

```

Input : studentAnswers[1..n],
        correctAnswers[1..n],
        questionWeight[1..n]
Output: score
1. score ← 0
2. for i ← 1 to n do
3.   if studentAnswers[i] ==
correctAnswers[i] then
4.     score ← score + questionWeight[i]
5.   end if
6. end for
7. return score

```

The shuffle algorithm used for question randomization is given in Algorithm 2. Each student attempt seeds a Fisher-Yates pass over the questions array, producing an unbiased permutation in $O(n)$ time and $O(1)$ extra memory.

Algorithm 2. Fisher-Yates Question Shuffle

```

Input : questions[1..n], rngSeed
Output: shuffledQuestions[1..n]
1. seedRandom(rngSeed)
2. for i ← n downto 2 do
3.   j ← randomInteger(1, i)
4.   swap(questions[i], questions[j])
5. end for
6. return questions

```

The RBAC middleware (Algorithm 3) is invoked on every protected route. It decodes the JWT, validates its signature against the server secret, and checks that the claimed role is in the allowed set declared by the route handler.

Algorithm 3. JWT-Based RBAC Middleware

```

Input : request, allowedRoles
Output: next() or HTTP 401/403
1. token ← request.header["Authorization"]
2. if token is null then return 401
3. payload ← verifyJWT(token, SERVER_SECRET)
4. if payload is invalid then return 401
5. if payload.role not in allowedRoles then
return 403
6. request.user ← payload
7. return next()

```

IV. IMPLEMENTATION

A. Technologies and Tools

The Examak prototype is implemented on the MERN stack. The front-end uses React.js 18 with React Router for client-side navigation, Tailwind CSS 3 for styling, and Axios for HTTP communication. The back-end uses Node.js 18 with Express.js 4 and follows a layered organization (routes, controllers, services, repositories). MongoDB 6 serves as the persistence engine, accessed via Mongoose 7. JWT (using the `jsonwebtoken` library) provides stateless authentication, and Bcrypt.js handles password hashing with a cost factor of 10. The AI extension is prototyped in Python 3.10 with Flask 2 [9]–[11].

B. Key Modules

The system is organized into eight functional modules:

(1) **Authentication and RBAC.** Users register and log in; passwords are hashed with Bcrypt before storage using a cost factor of 10 (approximately 100 ms per hash on commodity hardware, sufficient to resist offline brute-force without harming the user experience). On successful login, a JWT containing the user ID, role, and expiry is returned to the client and stored in an HTTP-only cookie. Express middleware validates the JWT signature and role claim on each protected request, in line with Algorithm 3.

(2) **User and Subject Management.** Administrators create and manage user accounts (with bulk import from CSV), assign roles, and maintain the catalogue of subjects available to instructors. Soft-deletion is used so that historical results remain auditable.

(3) **Question Bank.** Teachers create, edit, and tag questions by subject and difficulty level (Easy, Medium, Hard). Each question stores its type (MCQ or True/False), its options, its canonical answer, its weight, and optional metadata such as Bloom's taxonomy level for future analytics.

(4) **Exam Scheduling.** Teachers configure exam windows specifying start and end timestamps, total duration, the question selection policy (fixed list or random draw from the bank), and the randomization flag. A countdown timer on the client is reconciled with a server-side timestamp so that page refreshes or brief disconnections do not allow students to gain extra time, addressing one of the most common abuse vectors in browser-based examinations.

(5) **Auto-Grading Engine.** Implements Algorithm 1 with weighted scoring for objective items. Grading is performed server-side immediately upon submission and is idempotent: re-submission within the active window updates the most recent record without duplicating database entries.

(6) **Result and Analytics.** Generates per-student and per-exam analytics dashboards. Students see their own score, correct/incorrect breakdown, and percentile (if enabled by the instructor). Teachers see distributions, average scores per question, and identify questions that may need revision (e.g., questions answered correctly by very few students or by nearly all).

(7) **AI Question Generation.** A Flask service exposes an endpoint that accepts a subject/topic and returns generated question candidates produced by an NLP model. The Node.js back-end calls this endpoint over HTTPS with a shared secret, validates and sanitizes the returned JSON, and surfaces the candidates in the teacher's authoring UI for human review before they enter the question bank.

(8) **Anti-Cheating Controls.** The current release supports randomized question order (Algorithm 2), disabled context menu and copy/paste during exams (via React event handlers), a server-validated countdown timer, and basic tab-switch detection that logs visibility-change events. These controls are intentionally non-invasive; more aggressive proctoring (camera-based gaze tracking, full-screen lockdown) is deferred to the future AI-proctoring module.

C. Data Model

The MongoDB schema is organized around six principal collections: **users** (administrators, teachers, students, with role discriminator), **subjects** (catalogue of teachable units), **questions** (typed question bank with options,

canonical answers, weights, and difficulty), **exams** (schedule, configuration, references to the question set), **attempts** (per-student exam attempts with start and submission timestamps, randomized question order, and recorded answers), and **reports** (aggregated per-student or per-exam analytics). Indexes are declared on the foreign-key fields (**userId**, **examId**, **subjectId**) and on **createdAt** to accelerate the most common queries: dashboard rendering, history retrieval, and per-exam statistical aggregation.

D. API Design

The REST API follows resource-oriented conventions and is versioned under the `/api/v1` prefix. Representative endpoints include `POST /api/v1/auth/login`, `POST /api/v1/auth/register`, `GET /api/v1/users` (admin only), `POST /api/v1/subjects` (teacher and admin), `POST /api/v1/questions` (teacher), `POST /api/v1/exams` (teacher), `GET /api/v1/exams/upcoming` (student), `POST /api/v1/attempts/:examId/start`, `POST /api/v1/attempts/:attemptId/submit`, and `GET /api/v1/reports/student/:userId`. Every endpoint returns JSON and a uniform error envelope of the form `{ "error": string, "code": number, "details": object }`, which simplifies client-side handling.

E. Security Considerations

Security is enforced at four levels. *Transport*—all communication between the client, back-end, and Flask AI service is performed over HTTPS. *Authentication*—passwords are stored only as Bcrypt hashes; sessions are stateless JWTs with short expiry. *Authorization*—the RBAC middleware (Algorithm 3) enforces per-route role policies and additionally checks resource ownership (e.g., a student cannot fetch another student's attempt). *Input validation*—all incoming payloads are validated against Mongoose schemas and a request-validation layer that rejects malformed or oversized payloads early, mitigating injection and denial-of-service vectors.

F. Implementation Challenges

Several non-trivial engineering challenges were encountered. *Timer synchronization* across client refreshes was solved by storing the exam start time on the server and computing the remaining time on each render rather than relying on a client-only countdown; the server also rejects late submissions strictly. *Cross-language integration* between the Node.js back-end and the Python AI components was handled by exposing the AI module as a Flask micro-service and communicating over HTTPS with shared API keys. *Concurrency under load* was addressed via MongoDB connection pooling, stateless authentication (JWT), and

horizontal scalability of the Node.js back-end behind a reverse proxy. *Schema evolution*—adding new question types in the future without breaking existing data—was handled by tagging every document with a `schemaVersion` field that allows lazy migration on read.

V. TESTING AND EVALUATION

A. Testing Strategy

Examak was validated through four complementary testing approaches. *Unit testing* was applied to individual functions and components (login validation, scoring, question rendering) using Jest on the back-end and React Testing Library on the front-end. *Integration testing* exercised end-to-end flows that span multiple modules (e.g., create exam →

assign to subject → student attempts → auto-grade → store result). *Usability testing* was conducted with a small group of representative users across the three roles. *Stress testing* used load-generation scripts to simulate concurrent users and measure response time and error rate as load grew.

B. Performance Metrics

Quantitative performance results are summarized in Table II. The system met or exceeded every target. Auto-grading accuracy was 100%, as expected for a deterministic rule-based engine on objective items. Login and dashboard rendering averaged around 500 ms end-to-end. Exam submission, which involves writing answers, computing the score, and updating analytics, completed within 1.5 s under typical load.

TABLE II. PERFORMANCE EVALUATION RESULTS

Metric	Result	Target
Login Response Time	~500 ms	< 1 s
Dashboard Load Time	~500 ms	< 1 s
Exam Submission Time	< 1.5 s	< 2 s
Auto-Grading Accuracy	100%	100%
Concurrent Users Supported	300	≥ 200
UI Responsiveness	Good	Good
Cross-Browser Compatibility	Pass	Pass
Security (RBAC + JWT)	Validated	Validated

C. Stress Testing Methodology

Stress testing was conducted using a synthetic load-generation script that simulates concurrent students each running a full exam workflow: login, fetch upcoming exams, start an attempt, submit answers, and retrieve the result. Load was ramped from 10 to 300 concurrent virtual users in steps of 50 and held at each step for two minutes, while back-end response time, error rate, and CPU utilization were measured. At 300 concurrent users, the 95th-percentile latency for the submission endpoint remained below 1.5 s, the error rate stayed at 0%, and back-end CPU utilization peaked near 70% on the test hardware, indicating headroom for further growth.

D. Comparison with Existing Solutions

Compared with the platforms surveyed in Section II, Examak offers a more focused exam-lifecycle scope with a substantially smaller deployment footprint than Moodle or Blackboard. Unlike Google Forms, it provides explicit RBAC, JWT-based authentication, anti-cheating controls, and detailed analytics. Unlike Blackboard or ExamSoft, it is open and free, which lowers the barrier to adoption for smaller institutions and academic projects. Compared with custom B/

S-architecture solutions such as the one described in [3], Examak benefits from the modern, well-supported MERN ecosystem and from a clean extension seam for AI services.

E. Usability Findings

Usability feedback led to several refinements: clearer button labelling on the exam panel, a more prominent countdown timer, condensed teacher-side analytics with drill-down on demand, and consistent error messages on failed authentication. These changes shortened the average time to complete representative tasks (login, start an exam, review a student report) by an estimated 18–25% in informal trials. Student participants particularly appreciated the immediate display of results after submission, which is consistent with the findings of Ali *et al.* [4] and Kumar and Singh [5] on the value of instant feedback for learning.

VI. RESULTS AND DISCUSSION

A. Summary of Findings

Examak meets its core functional and non-functional requirements. It delivers automated, accurate grading for objective items, sub-second core operations, scalability up to

300 concurrent users, and tight integration of role-based security. It also provides a clean extension point for AI services through its Flask interface. Beyond raw performance, the project demonstrates that a focused, well-modeled MERN application can deliver an examination experience that rivals legacy LMSs for the specific scope of objective assessment, while remaining dramatically simpler to operate.

B. Architectural Insights

The three-tier model, although conventional, proved to be the right choice for this scale. A micro-services architecture would have introduced operational overhead disproportionate to the size of the system; a fully monolithic design would have made the AI extension awkward. Keeping the AI module behind a Flask micro-service preserved language-of-choice (Python) for the data-science work while leaving the operational core (Node.js) unchanged. This pattern—a small Python AI sidecar called from a Node.js core—is, in our experience, well-suited to other education-technology projects of comparable scope.

C. Interpretation of Results

The 100% grading accuracy is a direct consequence of using deterministic comparisons against canonical answer keys for objective items. The sub-second response times are attributable to three design decisions: MongoDB's indexed document model, stateless JWT-based authentication that removes per-request session lookups, and React's component-level reactivity that avoids full-page reloads. Stable behavior at 300 concurrent users in stress tests confirms that the chosen stack is appropriate for typical small-to-medium institutional deployments.

D. Limitations

Despite its strengths, the current Examak prototype has several limitations. First, only objective question formats are supported in the production-grade grader; subjective and essay-type items are not yet auto-graded. Second, AI-based proctoring (facial recognition, gaze tracking, behavior analysis, screen monitoring) is not implemented in the current release, though the architecture is prepared for it. Third, the system does not yet provide an offline examination mode, which would benefit students in low-connectivity regions. Fourth, two-factor authentication and additional hardening (e.g., refresh-token rotation, IP-based anomaly detection) are reserved for future work.

A further limitation concerns the scale of empirical evaluation. The 300 concurrent-user ceiling was determined by the test harness and not by an inherent constraint of the system; we expect that horizontal scaling of the Node.js back-end and a properly sized MongoDB cluster would extend this figure substantially, but such large-scale evaluation was outside the scope of this work. Likewise, the analytics module, while functional, does not yet provide cohort-level

longitudinal analysis or comparative benchmarking against external standards. These extensions are noted as planned future work.

E. Threats to Validity

As with any empirical evaluation, several threats to validity should be acknowledged. *Internal validity*: the testing was performed in a controlled environment that may not capture the long-tail behavior of production traffic. *External validity*: the system was evaluated with synthetic users and a small number of human participants; broader deployment may surface additional usability concerns. *Construct validity*: the chosen metrics (response time, accuracy, concurrent users) are standard but do not fully capture pedagogical effectiveness, which would require a longitudinal study correlating system use with learning outcomes.

F. Deployment Considerations

Examak is designed for straightforward deployment on commodity cloud infrastructure. A reference deployment uses Docker containers for the Node.js back-end and the Flask AI service, a managed MongoDB cluster (e.g., MongoDB Atlas) for the data layer, and a CDN-fronted static-site hosting service for the React build artefacts. Environment-specific configuration (database connection strings, JWT secrets, AI service URLs and shared keys) is supplied through environment variables and never committed to source control. A reverse proxy with TLS termination provides HTTPS and basic request rate limiting. This deployment topology has been validated on a single virtual machine with two virtual CPUs and 4 GB of RAM, which is sufficient for the 300-concurrent-user stress workload described above.

Operational concerns are addressed through structured logging, basic Prometheus-compatible metrics, and database backups. The audit trail captures authentication events, exam creation and modification, and submissions, which together provide enough information for post-hoc investigation of disputed exam attempts.

G. Pedagogical Implications

Beyond the technical results, this work has practical pedagogical implications. The combination of immediate feedback, secure delivery, and tamper-resistant timing reduces the gap between assessment and learning, which is a known driver of mastery [4], [5]. The detailed analytics surfaced to instructors enable per-question diagnostic review, supporting iterative refinement of the question bank and identification of mis-calibrated items. For institutions in resource-constrained settings, the low operational footprint of the platform translates into measurable cost savings compared to commercial alternatives, freeing budget for instructional design rather than software licensing.

VII. CONCLUSION AND FUTURE WORK

A. Conclusion

This paper presented Examak, an AI-enhanced web-based examination system implemented on the MERN stack with a clean three-tier architecture, JWT-based RBAC, Bcrypt password hashing, deterministic auto-grading for objective items, and an explicit Flask-based extension layer for AI services. The platform was modeled in detail using ERDs, DFDs, and UML diagrams (Figs. 1–6) and validated through unit, integration, usability, and stress testing. The results demonstrate 100% grading accuracy for objectives, sub-second core operations, and stable behavior up to 300 concurrent users, while comparing favorably against widely-used LMSs (Table I) on the dimensions of cost, deployment simplicity, and AI-readiness.

B. Broader Impact

A reliable, low-cost, AI-ready examination platform has measurable broader impact. For institutions in regions where commercial LMS licensing is prohibitive, an open MERN-based platform makes secure online assessment accessible. For instructors, the analytics surfaced by the system support evidence-based revision of question banks and identification of poorly calibrated items. For students, immediate, consistent, and bias-free grading on objective items reduces friction in the learning cycle. The architecture also positions the platform to absorb future AI capabilities without disruption, which is increasingly important as transformer-based language models become commodity infrastructure.

C. Lessons Learned

Several lessons emerged during the design and implementation of Examak. First, investing early in a clean three-tier separation paid off when the AI extension layer was introduced: no business-logic refactoring was required. Second, treating the timer as a server-side artifact rather than a client-side widget closed a substantial class of cheating vectors at very low engineering cost. Third, lightweight pseudocode-level specification of core algorithms (Algorithms 1–3) made code review and testing significantly easier than ad-hoc implementation. Fourth, even modest analytics—per-question correctness rates, per-student score histories—changed the conversation with instructors during usability sessions: they immediately began asking pedagogical questions rather than technical ones, which suggests that analytics are a primary lever for adoption.

D. Future Work

Several extensions are planned. (1) *AI-based proctoring* using facial recognition, gaze and behavior analysis, and screen monitoring to deter and detect cheating during high-stakes remote examinations. (2) *NLP-based automated essay*

grading, leveraging transformer-based language models exposed through the existing Flask interface. (3) *Automatic question generation* from a course syllabus or specified topics, with difficulty calibration. (4) *Mobile applications* implemented in React Native or Flutter to extend reach to smartphones and tablets. (5) *Offline-first mode*, allowing students in low-connectivity environments to take exams locally with secure synchronization on reconnect. (6) *Two-factor authentication* and refresh-token rotation to harden security. (7) *Advanced analytics dashboards* with cohort-level comparative analysis to support data-driven instructional decisions.

ACKNOWLEDGMENT

The author would like to express sincere gratitude to *Ir. Somantri, S.T., M.Kom* for his invaluable supervision, technical mentorship, and constructive feedback throughout this work, and to the Department of Informatics Engineering, Faculty of Engineering, Universitas Nusa Putra, for providing the academic environment and resources that made this research possible.

REFERENCES

1. A. Al-Azawei, P. Parslow, and K. Lundqvist, "Barriers and opportunities of e-learning implementation in Iraq: A case of public universities," *Int. Rev. Res. Open Distrib. Learn.*, vol. 18, no. 6, pp. 236–253, 2017.
2. H. N. Eke, "Modeling LIS students' intention to adopt e-learning: A case from University of Nigeria, Nsukka," *Library Philosophy and Practice (e-journal)*, 482, 2011.
3. L. Zhang, R. Zhao, and X. Wang, "Design and implementation of online examination system based on B/S architecture," *J. Phys. Conf. Ser.*, vol. 1631, no. 1, 2020.
4. R. Ali *et al.*, "Automated grading systems in higher education: An empirical study," *Int. J. Educ. Technol.*, 2020.
5. S. Kumar and A. Singh, "The impact of real-time feedback on student learning outcomes," *Comput. Educ.*, 2019.
6. Y. Chen *et al.*, "UI simplicity and online learning performance," *IEEE Trans. Learn. Technol.*, 2021.
7. Moodle Documentation. [Online]. Available: <https://docs.moodle.org>
8. Google Classroom. [Online]. Available: <https://classroom.google.com/>
9. React – A JavaScript library for building user interfaces. Meta. [Online]. Available: <https://reactjs.org>
10. Node.js Documentation. [Online]. Available: <https://nodejs.org/en/docs/>
11. MongoDB Documentation. [Online]. Available: <https://www.mongodb.com/docs/>
12. IBM Cloud Education, "What is artificial intelligence (AI)?," 2020. [Online]. Available: <https://www.ibm.com/cloud/learn/what-is-artificial-intelligence>
13. R. Durstenfeld, "Algorithm 235: Random permutation," *Commun. ACM*, vol. 7, no. 7, p. 420, 1964 (Fisher–Yates shuffle).
14. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, 2015.

15. D. Crockford, "The application/json media type for JavaScript Object Notation (JSON)," IETF RFC 4627, 2006.
16. E. Brewer, "CAP twelve years later: How the rules have changed," *IEEE Computer*, vol. 45, no. 2, pp. 23–29, 2012.